

Install Rust on Ubuntu

 linuxcapable.com/install-rust-on-ubuntu-linux

Joshua James

December 4, 2023



Ubuntu's own Rust packages are fine for stable, system-managed builds, but crates, editor tooling, and upstream examples move faster than Ubuntu's APT branch on most LTS releases. That gap matters when you install Rust on Ubuntu for active development work. Rust gives you the compiler, while Cargo handles new projects, dependencies, builds, and tests from the same terminal workflow.

Ubuntu 26.04, 24.04, and 22.04 all support the distro packages and the official rustup installer. Both documented methods install Cargo with the compiler: current default APT packages resolve to Rust and Cargo 1.93.x on Ubuntu 26.04 and 1.75.x on Ubuntu 24.04 and 22.04. rustup tracks the current upstream stable channel across every supported release, so it remains the better default for most active development work.

Update Ubuntu Before Installing Rust

Refresh APT metadata first, then choose whether Ubuntu or rustup should manage the toolchain on this system.

```
sudo apt update
```

These commands use `sudo` for system-wide package management. If your account does not have `sudo` access yet, run the commands as root or follow the guide to [add a new user to sudoers on Ubuntu](#).

Choose the Rust Installation Method on Ubuntu

Rust is available through Ubuntu's own packages and through the official upstream toolchain manager. APT keeps the compiler under Ubuntu's package database, while `rustup` tracks the current stable release and makes it easier to add nightly toolchains, editor components, and extra targets later. The Rust project does not publish an Ubuntu-specific `.deb` installer; for upstream-managed Linux installs, use `rustup`.

Method	Source	Branch	Updates	Best For
rustup	rustup.rs	Current stable channel	<code>rustup update</code>	Active development, current upstream docs, beta or nightly toolchains, editor tooling
Ubuntu APT	Ubuntu packages	Default Ubuntu package branch	APT-managed	System packages, conservative host maintenance, distro packaging parity

For most developers, `rustup` is the better fit because it stays aligned with the current stable release and the official Rust documentation. Ubuntu 26.04 and 24.04 also package `rustup` itself, but those repository versions trail the upstream manager and Ubuntu 22.04 does not ship `rustup` as an APT package, so the official installer script remains the most consistent `rustup` path across all supported Ubuntu LTS releases. If your search is really for Cargo, install Rust through `rustup` or use the APT method here; Ubuntu's `cargo` package depends on `rustc`, and `rustup` installs both commands together.

Ubuntu running inside WSL uses the same Linux `rustup` command from the Ubuntu terminal. Native Windows outside WSL should use [Rust's Windows installer](#) and the MSVC Build Tools instead of the Ubuntu commands.

Install Rust with rustup on Ubuntu

Use `rustup` when you want the current stable compiler, upstream documentation that matches your local toolchain, or the option to add nightly builds later without replacing Ubuntu packages by hand.

Install the prerequisite packages first. Standard Ubuntu desktops often already have certificates and the [curl command](#), but server, minimal, and custom images are the ones most likely to need the full set.

```
sudo apt install curl ca-certificates build-essential pkg-config
```

build-essential provides GCC, G++, and Make for crates that still compile native code during a build. pkg-config helps those crates find installed libraries such as OpenSSL or SQLite. If one of your projects also expects a generator step, [install CMake on Ubuntu](#) separately.

Run the official installer next:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
```

The --proto and --tlsv1.2 checks keep the download on HTTPS. The sh -s -- -y portion passes -y to the installer, so it accepts the default stable toolchain without waiting for an interactive menu. If you want to customize the profile or default toolchain, rerun the same command without -y.

Load Rust into the current shell so you can use it right away:

```
source "$HOME/.cargo/env"
```

The current shell needs this step once. New terminal sessions pick Rust up automatically after installation because rustup writes the same environment line into your shell startup configuration.

Then verify the toolchain and active channel. The exact version numbers change as Rust ships new stable releases, so treat the version lines as an example from the current stable channel:

```
rustc --version && cargo --version
rustup show active-toolchain

rustc 1.96.0 (ac68faa20 2026-05-25)
cargo 1.96.0 (30a34c682 2026-05-25)
stable-x86_64-unknown-linux-gnu (default)
```

rustup tracks the current stable release, so newer installs show the current upstream version instead of a fixed Ubuntu package branch.

Install Rust from Ubuntu APT

Choose the APT method when you want Ubuntu to manage Rust through normal package maintenance and you do not need the newest compiler branch on day one.

If a minimal or custom Ubuntu system returns Unable to locate package cargo, the missing piece is usually the Universe component. Enable it first with the guide to [enable Universe and Multiverse on Ubuntu](#), then rerun the install command.

```
sudo apt install rustc cargo
```

Check the versions Ubuntu installed:

```
rustc --version && cargo --version  
rustc 1.93.1 (01f6ddf75 2026-02-11) (built from a source tarball)  
cargo 1.93.1 (083ac5135 2025-12-15) (built from a source tarball)
```

Ubuntu 24.04 and 22.04 currently return rustc 1.75.0 and cargo 1.75.0 from the unversioned APT packages, even though APT metadata uses longer Ubuntu package revision strings. That version gap makes rustup the safer choice on 24.04 and 22.04 when your crates expect current Cargo behavior.

Build a Test Rust Project on Ubuntu

A quick project build proves more than a version check because it exercises Cargo, the compiler, and the local linker in one pass.

```
cargo new hello-rust --bin  
cd hello-rust  
cargo run --quiet
```

Hello, world!

Use cargo run --release when you want an optimized build instead of the default developer profile. If you want version control in the same workflow, [install and upgrade Git on Ubuntu](#) before you start committing Rust projects.

Add Rust Development Tools on Ubuntu

The default rustup profile already installs clippy and rustfmt. Add rust-analyzer when you want LSP-style completions, jump-to-definition support, and inline diagnostics in editors such as [Visual Studio Code on Ubuntu](#).

```
rustup component add rust-analyzer
```

Confirm the core development components are present:

The filter keeps the installed component list focused on the Rust tools most editors care about.

```
rustup component list --installed | grep -E 'rust-analyzer|clippy|rustfmt'  
clippy-x86_64-unknown-linux-gnu  
rust-analyzer-x86_64-unknown-linux-gnu  
rustfmt-x86_64-unknown-linux-gnu
```

If you stay on Ubuntu APT and only need the formatter plus the linter, install those packaged tools separately:

```
sudo apt install rustfmt rust-clippy
```

Ubuntu currently packages `rustfmt` and `rust-clippy`, but it does not package `rust-analyzer` for the supported Ubuntu LTS releases. If you need current editor components or matching upstream component updates, `rustup` stays cleaner than mixing packaged tools with an upstream-managed compiler.

Manage Rust Updates and Toolchains on Ubuntu

Rust maintenance depends on how you installed it, so use the matching update path instead of mixing `rustup` and APT commands.

Update Rust installed with `rustup` on Ubuntu

`rustup` refreshes every installed toolchain in one pass:

```
rustup update

info: syncing channel updates for 'stable-x86_64-unknown-linux-gnu'
info: checking for self-update
info: cleaning up downloads & tmp directories
```

If `stable` is already current, `rustup` ends by saying the `stable` toolchain is unchanged. If a newer release is available, it lists the downloaded and installed components instead.

Update Rust installed with Ubuntu APT

If you chose Ubuntu's packages, upgrade Rust, Cargo, and the optional packaged formatter and linter when you want a narrower change set than a full system upgrade:

```
sudo apt install --only-upgrade rustc cargo rustfmt rust-clippy
```

APT skips optional packages that are not already installed, so the same command can cover systems that never installed `rustfmt` or `rust-clippy`. Check `rustc --version` && `cargo --version` afterwards if you want to confirm that Ubuntu moved you to a newer packaged branch.

Install a nightly Rust toolchain on Ubuntu

The `nightly` toolchain is useful when a crate depends on unstable features or you want to test the next compiler line without replacing `stable`:

```
rustup toolchain install nightly
```

List the toolchains now available on your system:

```
rustup toolchain list

stable-x86_64-unknown-linux-gnu (active, default)
nightly-x86_64-unknown-linux-gnu
```

Use `cargo +nightly` build later when a single project needs nightly, and leave stable as the default for everything else.

Use rustup for Windows Targets from Ubuntu

Use this workflow only when you are already working in Ubuntu and need a Windows `.exe` build artifact. It does not replace the native Windows installer path; native Windows development should still use Rust's Windows installer and MSVC Build Tools.

Ubuntu's [Rust setup documentation](#) covers the same GNU target and MinGW linker family. Keep this path limited to Rust projects built from Ubuntu.

Install the GNU Windows linker package first:

```
sudo apt install gcc-mingw-w64-x86-64
```

Then add the Rust target with rustup:

```
rustup target add x86_64-pc-windows-gnu
```

Confirm the target is installed:

```
rustup target list --installed | grep -Fx x86_64-pc-windows-gnu
```

```
x86_64-pc-windows-gnu
```

From an existing Cargo project, build the Windows artifact with the target flag:

```
cargo build --target x86_64-pc-windows-gnu
```

Remove the optional target later if you only needed it for a one-off build:

```
rustup target remove x86_64-pc-windows-gnu
```

If no other cross-compilation work needs the linker, remove the Ubuntu package too:

```
sudo apt remove gcc-mingw-w64-x86-64
```

Fix Common Rust Issues on Ubuntu

Most fresh-install problems come down to shell PATH loading, mixed package methods, or missing CA certificates on stripped-down images.

Fix Rust command not found on Ubuntu

If the current shell still prints `bash: rustup: command not found`, `bash: rustc: command not found`, or `bash: cargo: command not found` right after a rustup install, check whether Rust's binary directory is already on your PATH.

```
case ":$PATH:" in
  *":$HOME/.cargo/bin:") echo "Rust PATH is loaded" ;;
  *) echo "Rust PATH is missing" ;;
esac
```

Rust PATH is loaded

If that command prints Rust PATH is missing, load the environment file in the current shell:

```
source "$HOME/.cargo/env"
```

Then verify the toolchain again:

```
rustc --version && cargo --version
```

Fix APT and rustup version conflicts on Ubuntu

When both installation methods are present, the shell can keep using Ubuntu's older `/usr/bin/rustc` even after rustup installs a newer toolchain under `~/.cargo/bin`. If you want a broader explanation of PATH resolution, see the [which command guide](#).

```
rustc_path=$(command -v rustc)
case "$rustc_path" in
  "$HOME/.cargo/bin/rustc") echo "rustup rustc is first on PATH" ;;
  /usr/bin/rustc) echo "APT rustc is first on PATH" ;;
  *) echo "unexpected rustc path: $rustc_path" ;;
esac
rustup show active-toolchain
```

```
rustup rustc is first on PATH
stable-x86_64-unknown-linux-gnu (default)
```

If the command lookup prints APT rustc is first on PATH instead, Ubuntu's APT packages are still winning. Remove the APT packages and reload the shell if you want rustup to stay in control:

```
sudo apt remove rustc cargo rustfmt rust-clippy && source "$HOME/.cargo/env"
```

Fix rustup SSL certificate errors on Ubuntu

If the installer cannot validate HTTPS, Ubuntu is usually missing current CA certificates.

```
curl: (60) SSL certificate problem: unable to get local issuer certificate
```

Reinstall the certificate bundle first:

```
sudo apt install --reinstall ca-certificates
```

Then confirm the installer URL returns a normal HTTPS response:

```
curl -fsSI https://sh.rustup.rs | head -n 1
```

Remove Rust from Ubuntu

Use the removal path that matches the method you installed. rustup and APT manage different files, so one command does not fully clean up the other.

Remove Rust installed with rustup on Ubuntu

This command removes the rustup-managed toolchains, the Cargo home, and the shell startup lines rustup added. The `-y` flag skips the confirmation prompt.

```
rustup self uninstall -y

info: removing rustup home
info: removing cargo home
info: removing rustup binaries
info: rustup is uninstalled
```

rustup leaves project directories such as `~/hello-rust` alone, so remove those separately if you no longer want them. It does remove the `.$HOME/.cargo/env` line from the shell startup files it changed during installation.

Verify that the toolchain directories are gone:

```
for name in .cargo .rustup; do
  path="$HOME/$name"
  [ -e "$path" ] && echo "still present: ~/$name" || echo "removed: ~/$name"
done

removed: ~/.cargo
removed: ~/.rustup
```

Remove Rust installed from Ubuntu APT

APT users can remove the compiler, Cargo, and optional packaged Rust tools with the standard package command:

```
sudo apt remove rustc cargo rustfmt rust-clippy
```

If APT now marks compiler dependencies as unused and you no longer need them for other projects, preview the cleanup list first:

```
sudo apt autoremove --dry-run
```

If the dry run lists only Rust-related dependencies you no longer need, run the cleanup without `--dry-run`:

```
sudo apt autoremove
```

Confirm that the APT Rust packages are no longer installed:

```
dpkg-query -W -f='${db:Status-Abbrev} ${Package}\n' rustc cargo rustfmt rust-clippy  
2>/dev/null | grep '^ii' || echo "Rust APT packages are not installed"
```

Rust APT packages are not installed

Conclusion

Rust is installed on Ubuntu with either the distro packages or the current stable toolchain from rustup. Start versioning your crates with [Git on Ubuntu](#), pair editor support with [Visual Studio Code on Ubuntu](#), and keep [The Rust Book](#) nearby as you move from hello-world binaries to larger services.